PROJECT DOCUMENT ON
SMART PARKING SYSTEM USING GRAPH TRAVERSAL

(DESIGN AND ANALYSIS OF ALGORITHMS)

By

Roll number	Name
A24126510122	BONDA GEETESH
A24126510132	GUJJALA CHANDU
A24126510142	KOTANA JOSHNAVI
A24126510152	PATNAIKUNI ADARSH
A24126510154	PENTAKOTA DESHIK
A24126510168	TEKKAMU JOSHITHA

SUBMITTED TO

Mrs. Botcha Vineela Rani (M.Tech,Ph.D)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES (A+)

(*Affiliated to Andhra University)*

Sangivalasa 531162, Bheemunipatnam mandal, Visakhapatnam Dist.

Abstract:-

This project presents a smart and efficient solution for parking management using the concept of Graph Traversal Algorithm from Design and Analysis of Algorithms (DAA). Parking management is one of the most common real-world problems in urban areas such as colleges, hospitals, shopping malls, railway stations, apartment complexes, and office buildings. In traditional parking systems, users often waste time and effort searching for a free parking slot manually. This leads to traffic congestion, improper space utilization, and confusion among drivers.

To solve this problem, the proposed system models the parking area as a graph, where each parking slot is represented as a vertex (node) and the connection between neighboring slots is represented as an edge. The system uses the Breadth First Search (BFS) algorithm to search for the nearest available parking slot from the entrance. BFS is an efficient graph traversal technique that helps in finding the closest free slot in an organized and systematic way.

The project is implemented using Java and Java Swing GUI. The system allows the user to enter a vehicle number, park a car, remove a parked car, and display the current status of all parking slots. In addition to parking allocation, the system also manages a waiting queue for vehicles when all slots are occupied. Once a parked vehicle leaves, the next vehicle in the queue is automatically assigned to the nearest free slot.

The proposed smart parking system improves parking efficiency, reduces time wastage, simplifies parking management, and demonstrates a practical application of graph traversal algorithms in real-world scenarios. Overall, the project is simple, effective, user-friendly, and highly relevant to modern smart parking needs.

Table Of Contants:-

S.no	Sections	Page no:
1.	Introduction	4
2.	Implementation	5-9
3.	Sample Inputs & Outputs	9-11
4..	Conclusion	12
5.	Future Scope	12
6.	References	13

Introduction:-

Parking management has become an important problem in today's world due to the rapid increase in the number of vehicles. In places such as colleges, malls, hospitals, offices, apartments, and public parking areas, finding a suitable parking slot has become difficult and time-consuming.

Most traditional parking systems are manual, unorganized, and inefficient, causing confusion and delay for drivers.

To solve this problem, a Smart Parking System Using Graph Traversal Algorithm is proposed. In this system, parking slots are represented using a graph structure, where each parking slot is treated as a node and the

connections between slots are treated as edges. The system uses the Breadth First Search (BFS) algorithm to identify the nearest available parking slot from the entrance point.
This approach helps in reducing the time required to search for a parking space and provides a systematic way of managing vehicle entry and exit. The project also includes a waiting queue to handle situations when all parking slots are occupied. The entire system is implemented using Java with a Graphical User Interface (GUI) for easy interaction.

Problem Background:-

- Manual parking slot allocation is time-consuming and inefficient
- Drivers often waste time searching for available parking spaces
- Difficulty in finding the nearest free parking slot quickly
- High chance of traffic congestion inside parking areas
- Inefficient utilization of available parking slots
- Lack of proper management when parking slots are full
- No automated system to handle incoming and outgoing vehicles effectively
- Traditional parking systems do not use intelligent algorithms for slot allocation

Importance:-

- Reduces the time required to find a parking slot
- Minimizes fuel wastage and unnecessary vehicle movement
- Provides a systematic and automated parking allocation process
- Ensures better utilization of all available parking slots
- Reduces confusion and congestion inside parking areas
- Improves reliability and efficiency of parking management
- Supports scalability for larger parking areas
- Demonstrates practical application of graph traversal algorithms
- Helps in building smart and organized parking infrastructure

Objectives:-

- Automate the parking slot allocation process
- Find the nearest available parking slot using BFS
- Represent the parking lot using a graph data structure
- Reduce manual effort in parking management
- Improve parking efficiency and space utilization
- Maintain a waiting queue when parking is full
- Provide a user-friendly GUI-based system
- Ensure easy implementation and practical usability

Implementation:-

The Smart Parking System Using Graph Traversal Algorithm is implemented using Java and Java Swing GUI. The system is designed to manage parking slot allocation intelligently by finding the nearest available parking slot using the Breadth First Search (BFS) algorithm.

1. ParkingSystem.java

This file contains the core logic of the system such as:

- parking slot initialization
- graph creation
- BFS traversal
- parking a car
- removing a car
- waiting queue management
- displaying parking status

2. SmartParkingGUI.java

This file contains the graphical user interface of the project using Java Swing.
It provides:

- input field for entering car number
- buttons for parking operations
- output display area
- user interaction support

1. ParkingSystem.java

```
import java.util.*;

class ParkingSystem {

    int capacity = 10;
    String[] slots = new String[capacity];

    HashMap<String, Integer> map = new HashMap<>();
    Queue<String> waitingQueue = new LinkedList<>();

    // Graph representation
    List<List<Integer>> graph = new ArrayList<>();

    ParkingSystem() {
        // Initialize slots
        for (int i = 0; i < capacity; i++) {
slots[i] = null;
            graph.add(new ArrayList<>());
        }

        // Create simple linear connections        for
(int i = 0; i < capacity - 1; i++) {

            graph.get(i).add(i + 1);

             graph.get(i + 1).add(i);
        }
    }

    // BFS to find nearest free slot        private int
findNearestSlot() {              boolean[] visited = new
boolean[capacity];
        Queue<Integer> q = new LinkedList<>();

        int entrance = capacity / 2; // middle slot
q.add(entrance);        visited[entrance] = true;

        while (!q.isEmpty()) {
            int current = q.poll();

            if (slots[current] == null) {
return current;
            }
```

```
            for (int neighbor : graph.get(current)) {
if (!visited[neighbor]) {


                    visited[neighbor] = true;
q.add(neighbor);
                }
            }
        }

        return -1;
    }

    public String parkCar(String carNumber) {

        if (map.containsKey(carNumber)) {
            return "Car already parked!";
        }

        int slot = findNearestSlot();

        if (slot != -1) {
            slots[slot] = carNumber;            map.put(carNumber,
slot);
            return " Car " + carNumber + " parked as slot " + slot + " (BFS Nearest) ";
        }
        waitingQueue.add(carNumber);
        return "Parking Full! Added to waiting queue.";
    }
    public String removeCar(String carNumber) {

        if (!map.containsKey(carNumber)) {
return "Car not found!";
        }

        int slot = map.get(carNumber);
        slots[slot] = null;
        map.remove(carNumber);

        if (!waitingQueue.isEmpty()) {
            String nextCar = waitingQueue.poll();
            int newSlot = findNearestSlot();

            if (newSlot != -1) {
slots[newSlot] = nextCar;
map.put(nextCar, newSlot);
                return "Removed. " + nextCar + " parked at slot " + newSlot;
```

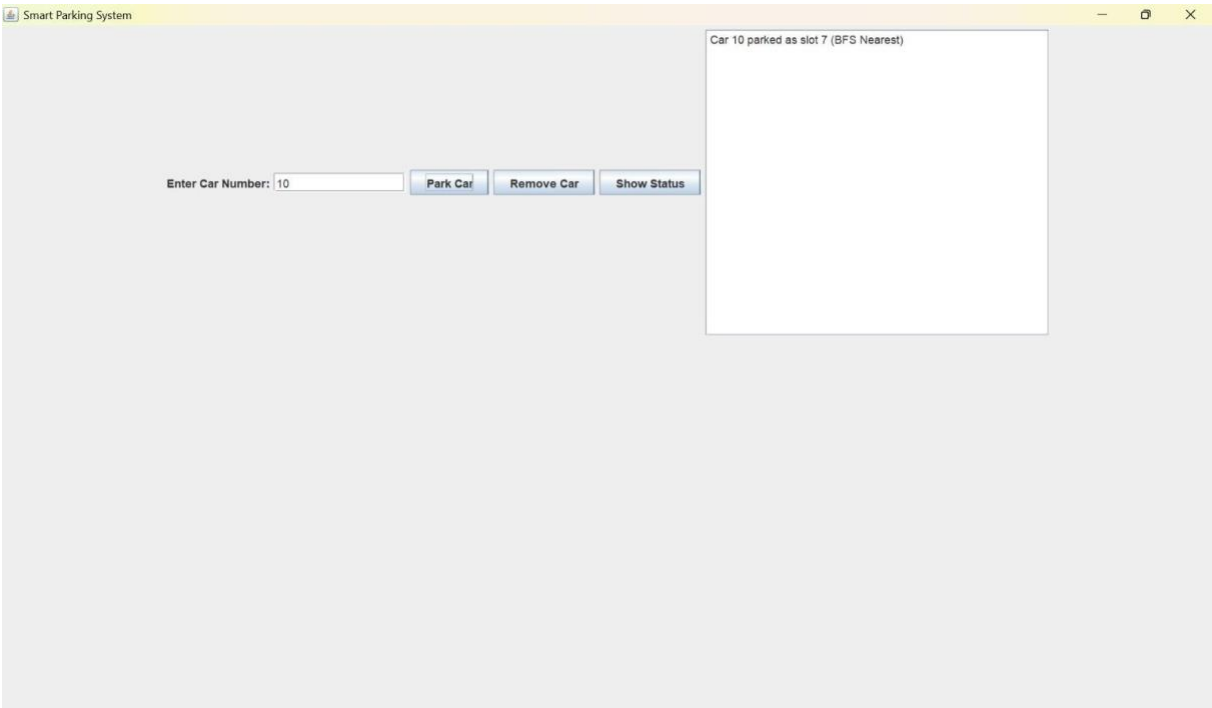
```
            }
        }

        return "Car removed from slot " + slot;
    }

    public String display() {

        StringBuilder result = new StringBuilder();

        for (int i = 0; i < capacity; i++) {            result.append("Slot
").append(i).append(": ")                    .append(slots[i] == null ?
"Free" : slots[i])
                .append("\n");
        }

        result.append("\nWaiting Queue: ").append(waitingQueue);

        return result.toString();
    }
}
```

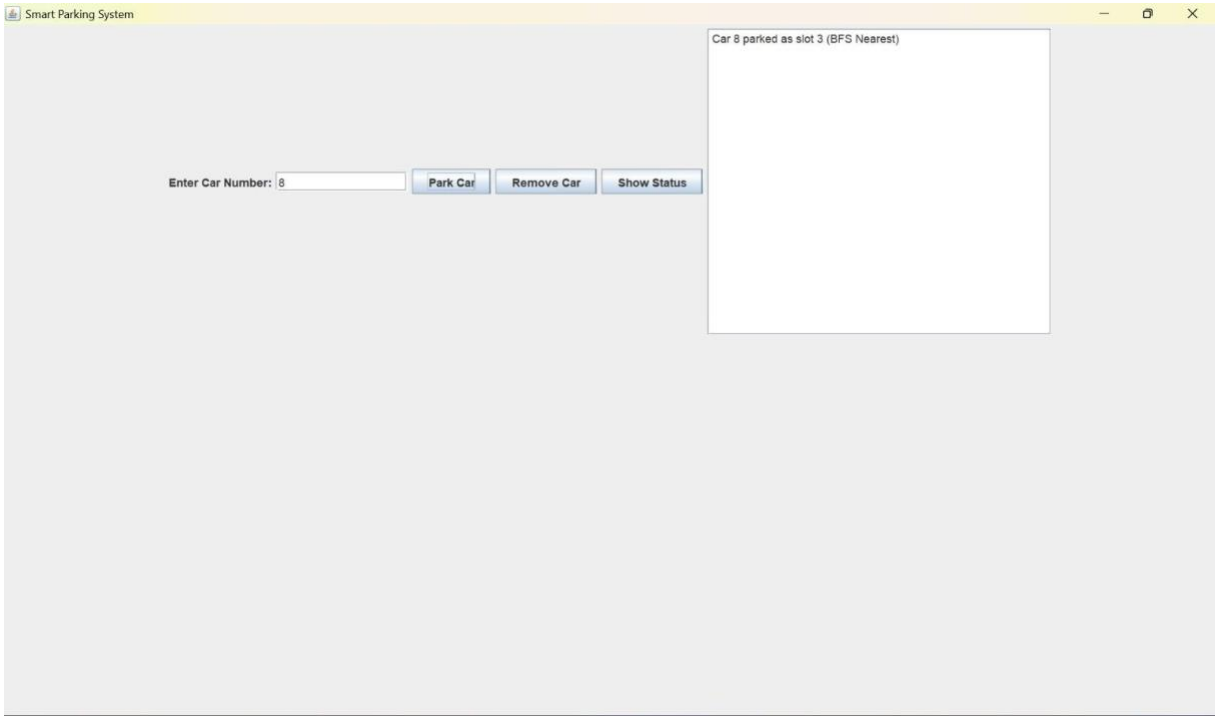
2. SmartParkingGUI.java:

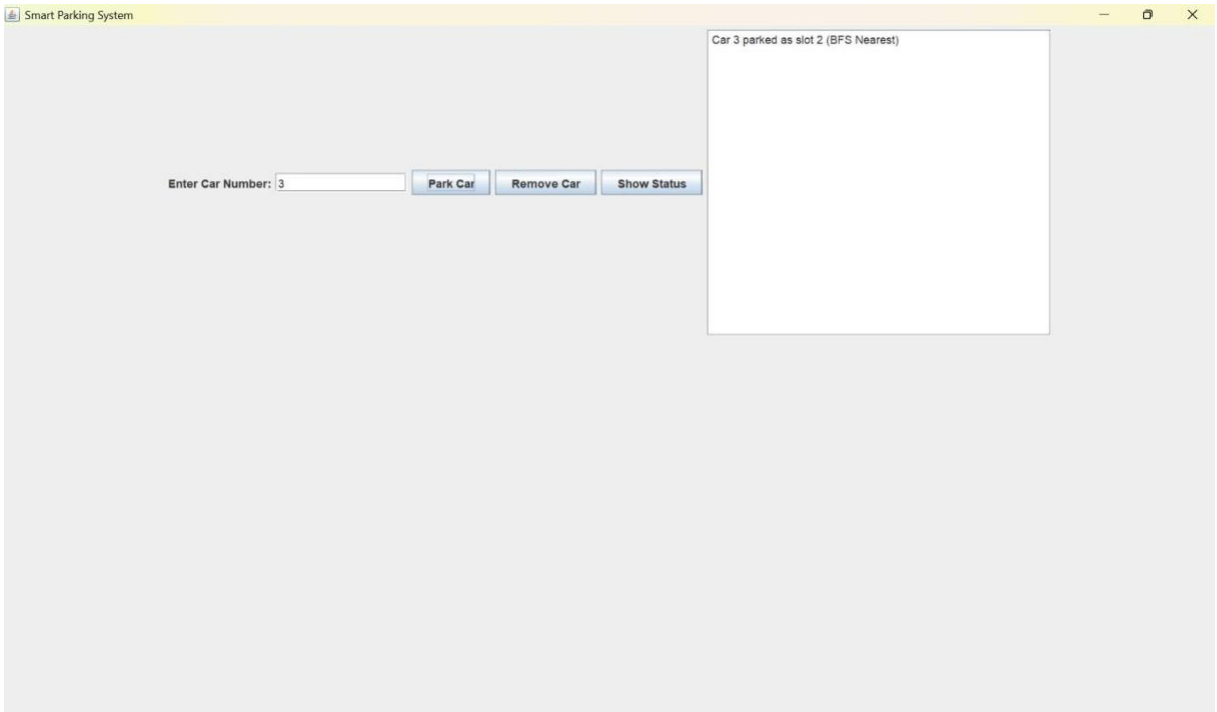
```
import javax.swing.*;
import java.awt.*;

public class SmartParkingGUI {      public static void
main(String[] args) {            ParkingSystem ps = new
ParkingSystem();

        JFrame frame = new JFrame("Smart Parking System");
        frame.setSize(500, 500);
        frame.setLayout(new FlowLayout());

        JTextField carField = new JTextField(15);        JTextArea
output = new JTextArea(20, 40);
        output.setEditable(false);

        JButton parkBtn = new JButton("Park Car");
        JButton removeBtn = new JButton("Remove Car");
        JButton showBtn = new JButton("Show Status");

        frame.add(new JLabel("Enter Car Number:"));
        frame.add(carField);        frame.add(parkBtn);
frame.add(removeBtn);        frame.add(showBtn);
```

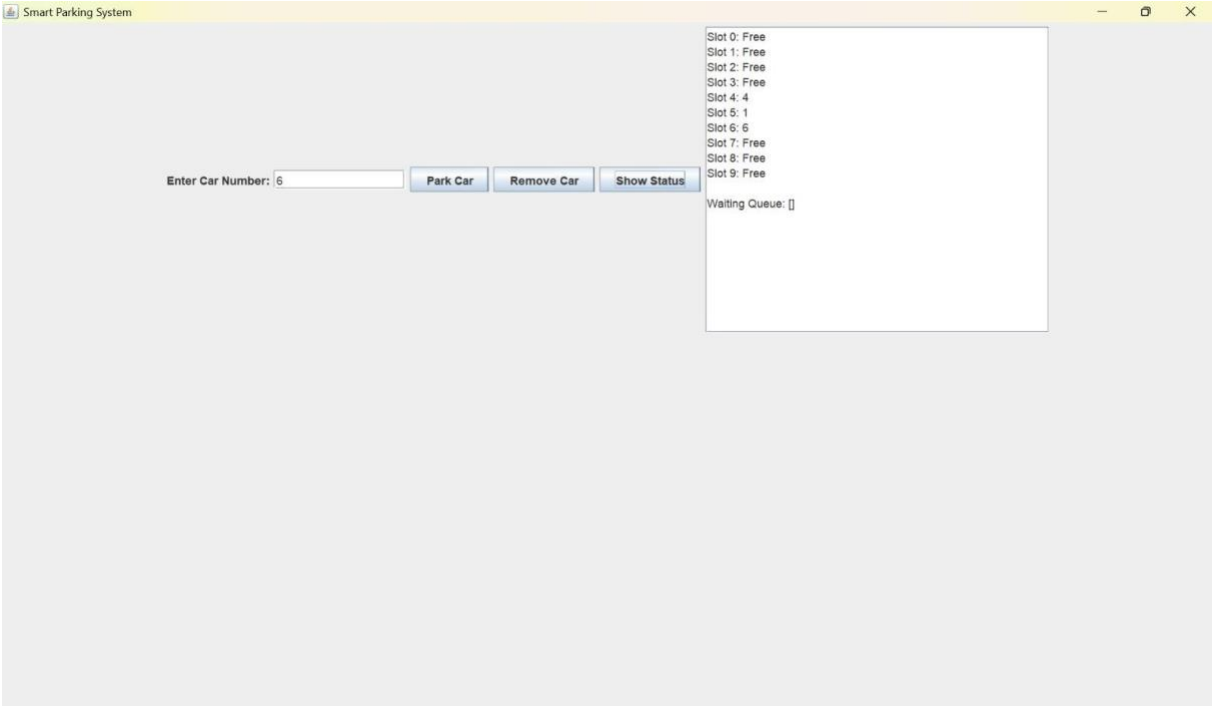
```
        frame.add(new JScrollPane(output));

        parkBtn.addActionListener(e -> {
String car = carField.getText();
            output.setText(ps.parkCar(car));
        });

        removeBtn.addActionListener(e -> {
String car = carField.getText();
            output.setText(ps.removeCar(car));
        });

        showBtn.addActionListener(e -> {            output.setText(ps.display());
        });

        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

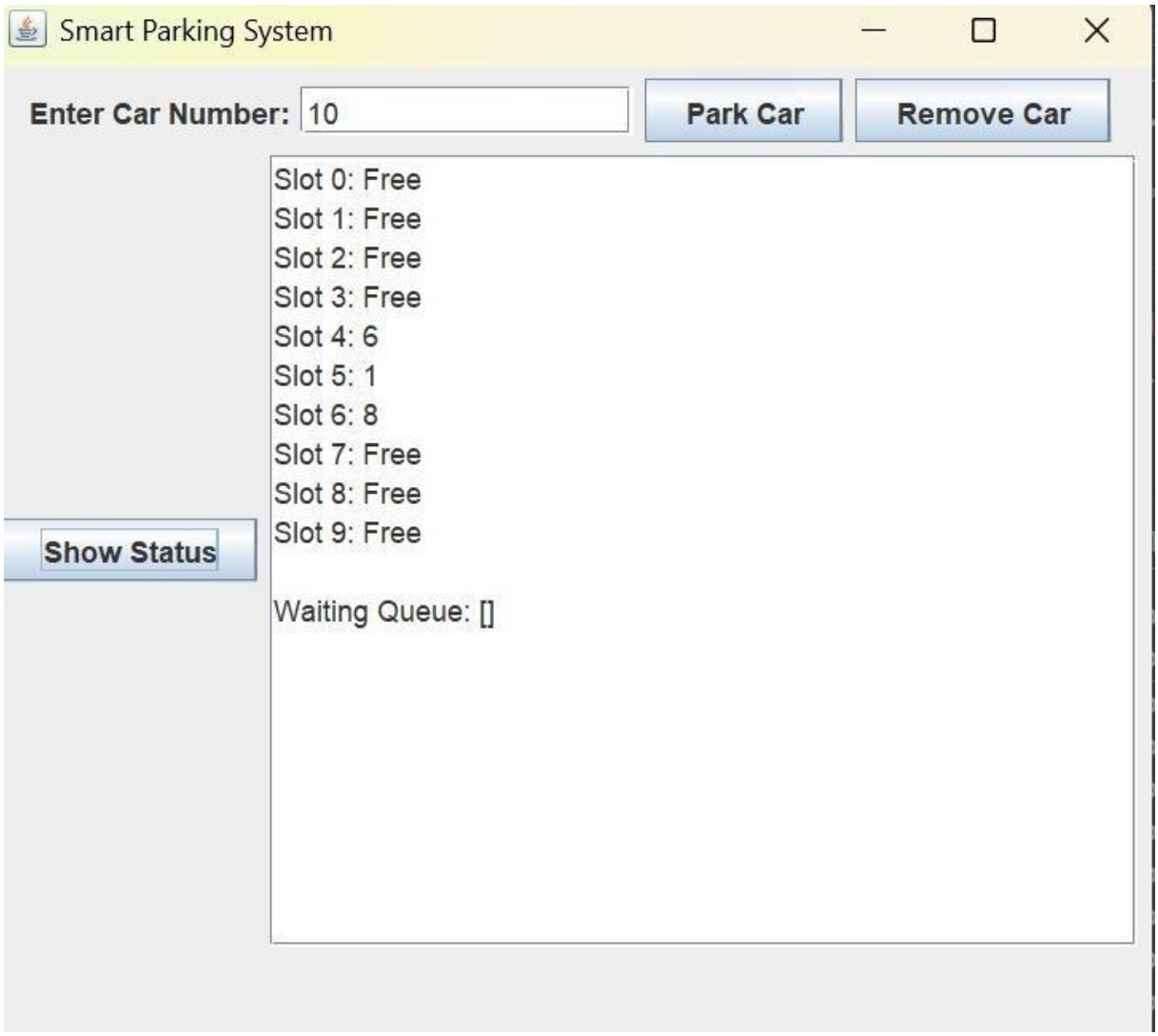
Sample Input & Output :-




Smart Parking System
Car 8 parked as slot 3 (BFS Nearest)
Enter Car Number: 8
Park Car
Remove Car
Show Status


Smart Parking System
Car 3 parked as slot 2 (BFS Nearest)
Enter Car Number: 3
Park Car
Remove Car
Show Status


Smart Parking System
Enter Car Number: 6
Park Car
Remove Car
Show Status
Slot 0: Free
Slot 1: Free
Slot 2: Free
Slot 3: Free
Slot 4: 4
Slot 5: 1
Slot 6: 6
Slot 7: Free
Slot 8: Free
Slot 9: Free
Waiting Queue: []


Smart Parking System
Enter Car Number: 10
Park Car
Remove Car
Show Status
Slot 0: Free
Slot 1: Free
Slot 2: Free
Slot 3: Free
Slot 4: 6
Slot 5: 1
Slot 6: 8
Slot 7: Free
Slot 8: Free
Slot 9: Free
Waiting Queue: []

Conclusion :-

The Smart Parking System Using Graph Traversal Algorithm is an efficient and practical solution for managing parking slots in an organized manner. This project successfully demonstrates the use of graph theory and the Breadth First Search (BFS) algorithm to identify the nearest available parking slot from the entrance point.
The system helps in reducing the time and effort required to search for parking spaces manually.

It improves parking slot utilization, avoids unnecessary vehicle movement, and provides a more systematic way of handling vehicle entry and exit. The addition of a waiting queue also makes the system more effective in handling situations when all parking slots are occupied.

The implementation of the project using Java and Java Swing GUI makes it simple, user-friendly, and easy to understand. Users can park a car, remove a car, and view the current parking status through a graphical interface. Overall, this project proves that graph traversal algorithms can be applied effectively to solve real-world parking management problems. It is a simple yet meaningful application of Design and Analysis of Algorithms (DAA) concepts in a practical scenario.

Future Scope :-

The proposed Smart Parking System Using Graph Traversal Algorithm can be further improved and extended in many ways to make it more useful for realworld applications. Although the current system provides an efficient solution for parking slot allocation, several advanced features can be added in the future to enhance its functionality and scalability.

Some possible future enhancements are:

- Integration with real-time parking sensors to automatically detect free and occupied slots
- Development of a mobile application for easier user access
- Implementation of online parking slot booking / reservation system
- Addition of database connectivity to store parking records permanently
- Integration of parking fee calculation and digital payment system
- Support for multi-floor parking management
- Use of GPS-based navigation to guide the user to the allocated parking slot
- Implementation of QR code / RFID-based vehicle entry and exit system
- Use of more advanced graph algorithms for better optimization

- Integration with smart city infrastructure and IoT-based systems

Thus, the project has good future potential and can be developed into a more intelligent and fully automated smart parking solution.

References :-

1. Java Programming Language Documentation
2. Data Structures and Algorithms using Java
3. Introduction to Algorithms
4. Graph Traversal Algorithms – BFS and DFS
5. Java Swing GUI Concepts
6. Smart Parking System Research Papers
7. Design and Analysis of Algorithms Study Material